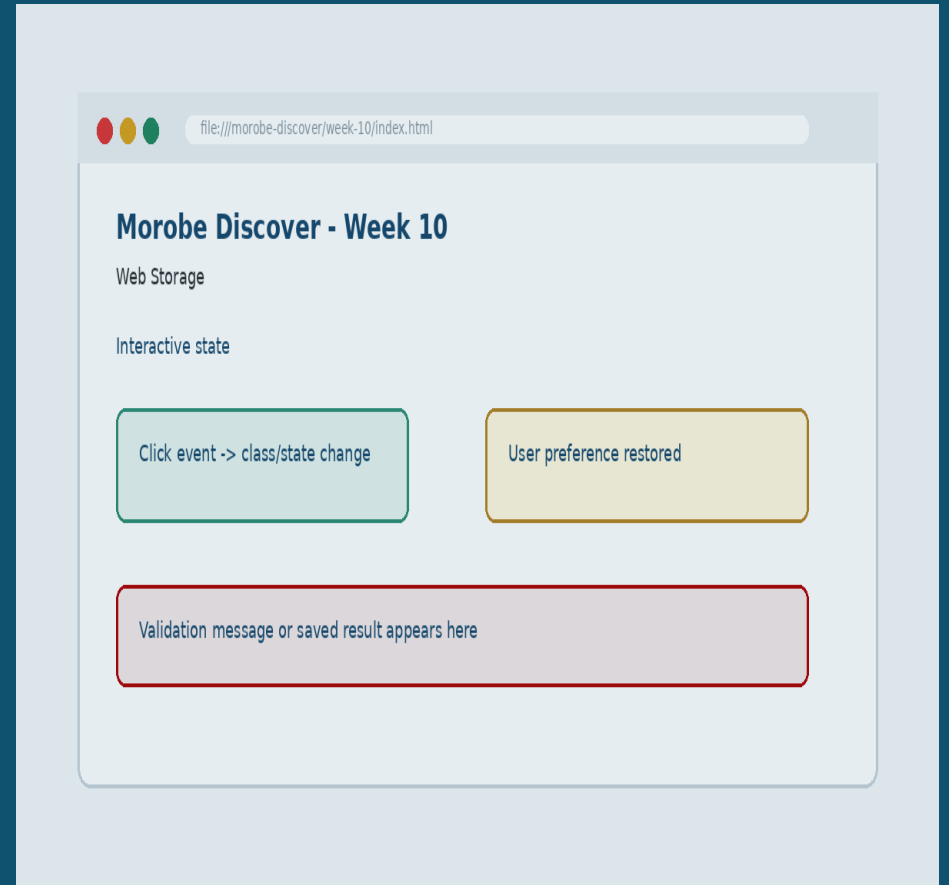


Week 10

Web Storage

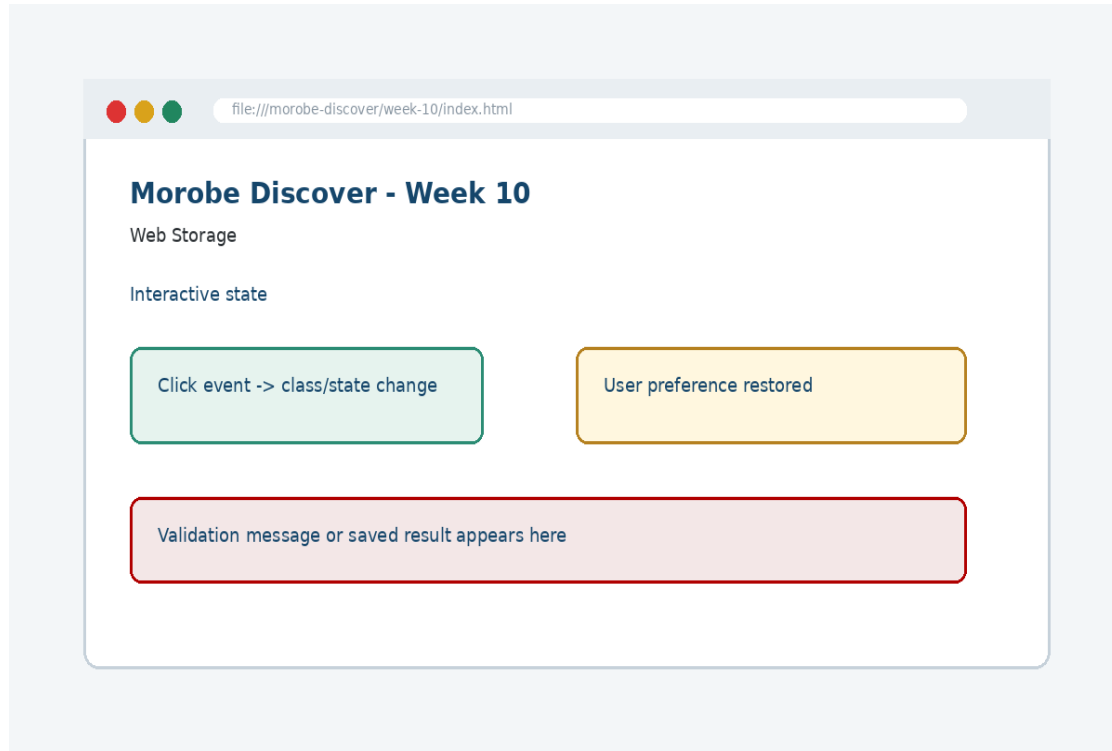
Persist lightweight user preferences between browser sessions.



Learning outcomes

Persist lightweight user preferences between browser sessions.
Explain the core programming/design concept for this week
Read and modify key code patterns
Test the weekly project increment
Prepare evidence for GitHub submission

Today's problem



Why it matters

Browser storage is useful for convenience, but it must be small, safe and recoverable.

Project before -> after

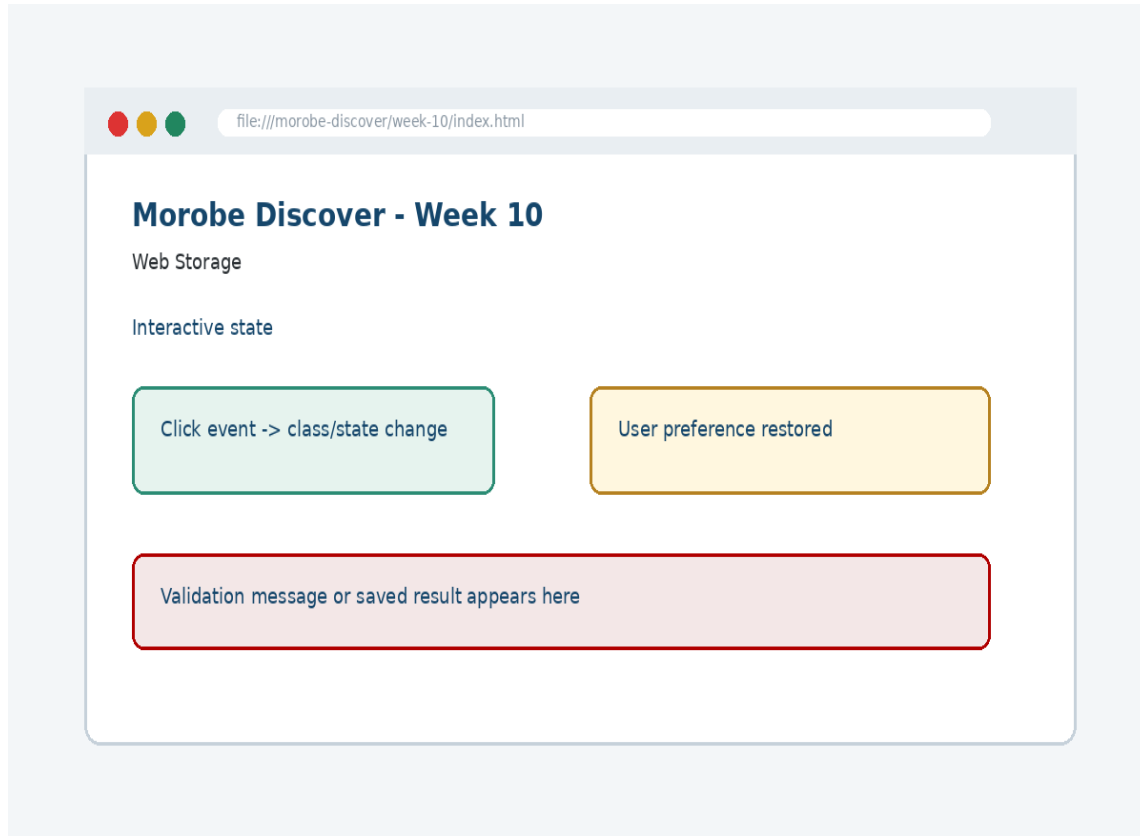
Before Week 10

Week 09 identified usability and accessibility issues in the interactive site.

After Week 10

The site remembers non-sensitive user preferences and restores them predictably.

What we build this week



Review current storage use
Store favourite destination IDs
Restore saved theme on page load
Handle missing or corrupted storage safely
Provide a clear reset option

Core principle

Browser storage is useful for convenience, but it must be small, safe and recoverable.

Keep in mind

A good web developer can point to the code, the browser result, and the user reason behind the decision.

Key concepts

Concept 1

localStorage persists after the browser closes

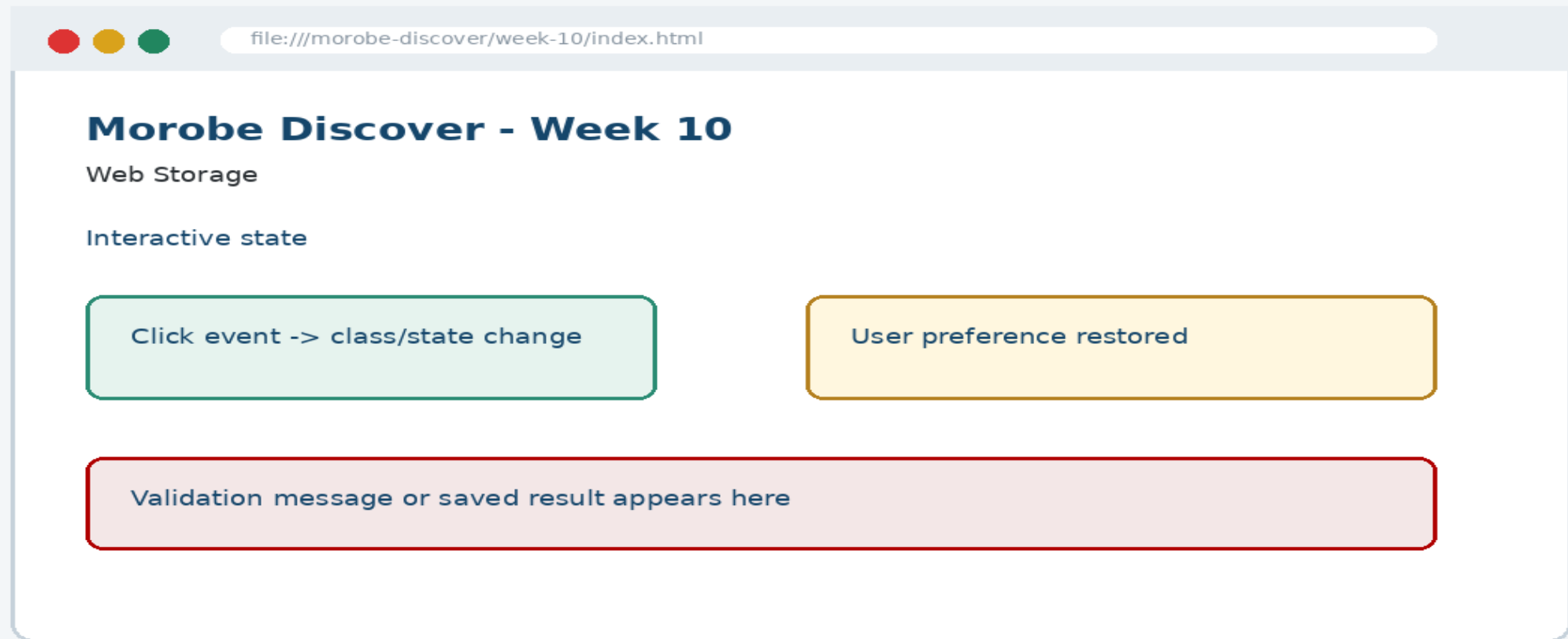
Concept 2

sessionStorage lasts only for a session

Concept 3

Storage values are strings and should be parsed carefully

Project anatomy



Key code pattern A

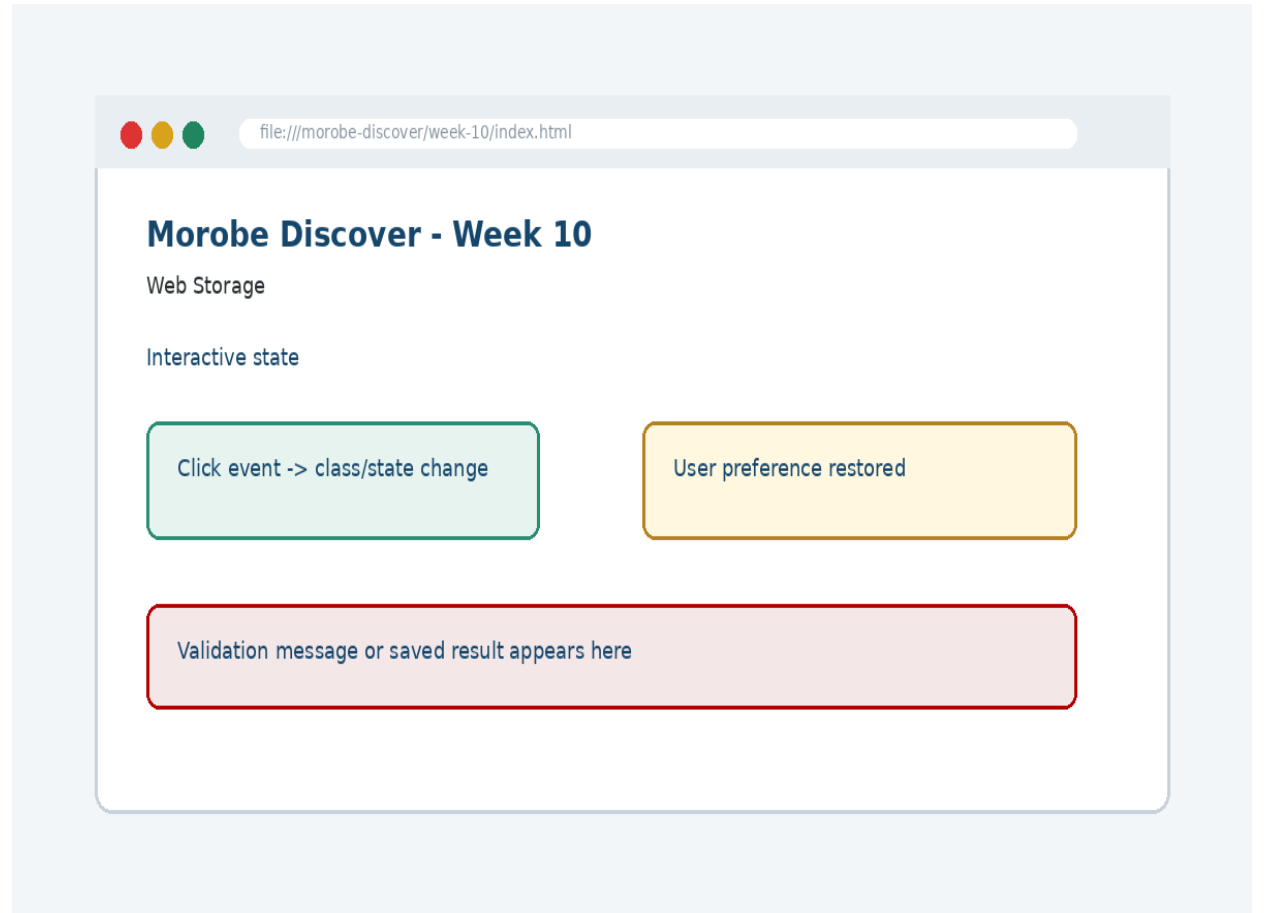
```
const favourites =  
JSON.parse(localStorage.getItem("favourites") ||  
"[]");  
favourites.push("rainforest-walk");  
localStorage.setItem("favourites",  
JSON.stringify(favourites));
```

Explain it

Which file contains this code?
Which line creates the visible change?
What should appear in the browser?

Code -> visual result

```
const favourites =  
JSON.parse(localStorage.getItem("favourite  
s") || "[]");  
favourites.push("rainforest-walk");  
localStorage.setItem("favourites",  
JSON.stringify(favourites));
```



What changed?

Project file changed for Web Storage

The browser output is now closer to the required weekly evidence

The change supports a user task or quality requirement

The change can be tested and committed

Key code pattern B

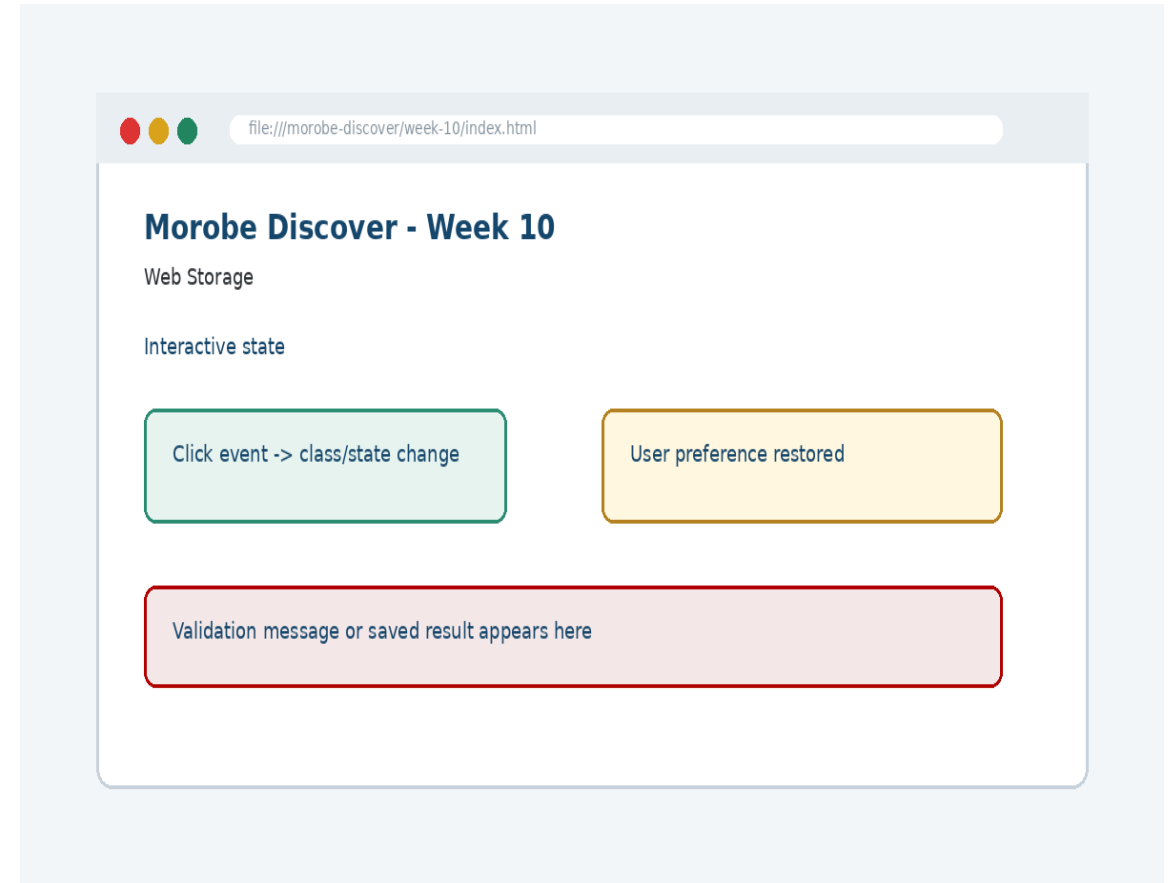
```
try {  
  const data = JSON.parse(savedValue);  
} catch {  
  localStorage.removeItem("favourites");  
}
```

Why this pattern is needed

The second pattern usually applies the same idea to a different component, state, layout or data flow.

Apply it to Morobe Discover

Review current storage use
Store favourite destination IDs
Restore saved theme on page load
Handle missing or corrupted storage safely



Compare the approaches

Weak approach

Memorise the definition
Copy code without understanding
No browser evidence
No test note

Professional approach

Explain the concept
Apply the correct code pattern
Inspect the result
Commit and describe the evidence

Common mistake

Mistake

The page appears unchanged, but the student assumes the feature is complete.

Correction

Check the correct file, refresh the browser, inspect DevTools, and verify the expected output.

Why does this fail?

```
// Broken checkpoint  
// File changed, but the browser result does not  
change.  
// Possible causes:  
// - wrong file  
// - file not linked  
// - syntax error  
// - browser not refreshed
```

Class question

Which cause would you check first, and why?

Fix the problem

- Confirm the browser is opening the correct file
- Check links to CSS/JS/data files
- Inspect the console for errors
- Use a small working example
- Test again before committing

Class activity

Activity - 7 minutes

Compare localStorage and sessionStorage for a trip-planning draft.

Work in pairs

Point to the project evidence

Prepare one answer to share

Implementation steps 1-3

Review current storage use
Store favourite destination IDs
Restore saved theme on page load

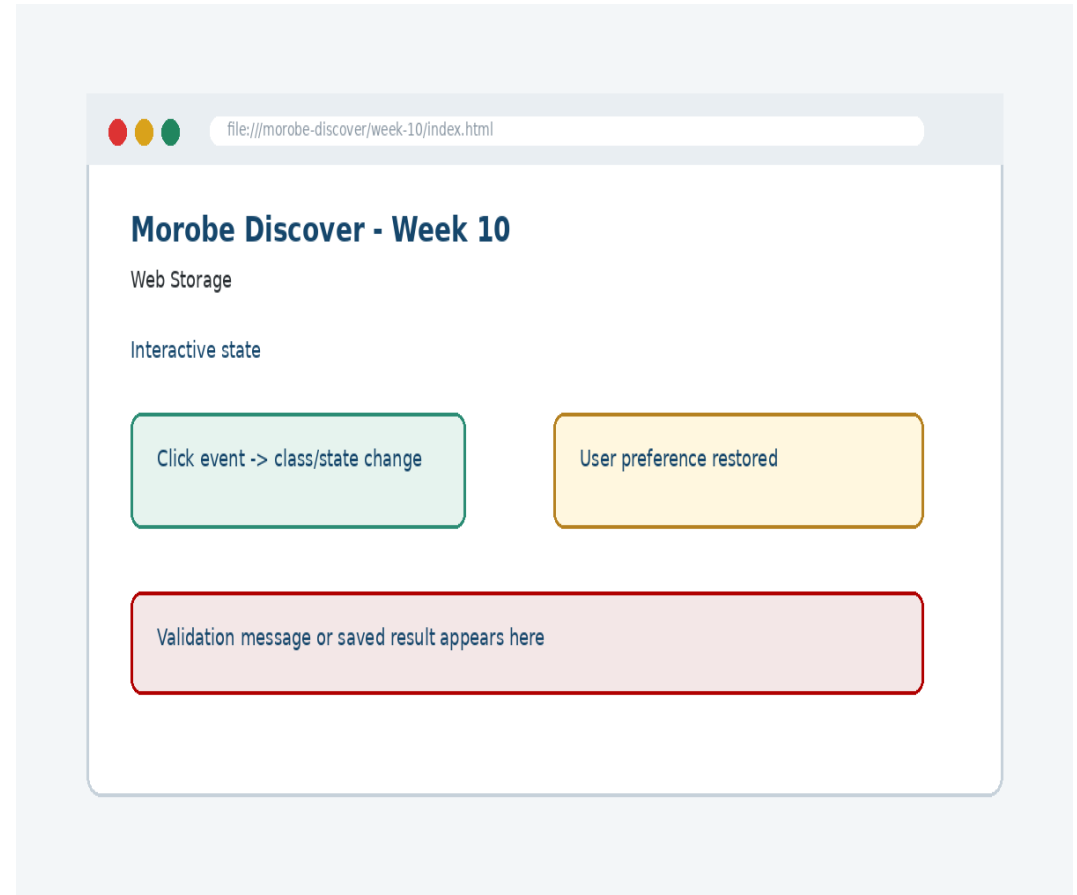
Implementation steps 4-5

Handle missing or corrupted storage safely
Provide a clear reset option

LIVE DEMO

Demo checklist

1. Open the current project
2. Locate the affected file
3. Add or modify the key code
4. Refresh the browser
5. Inspect the result
6. Explain the change



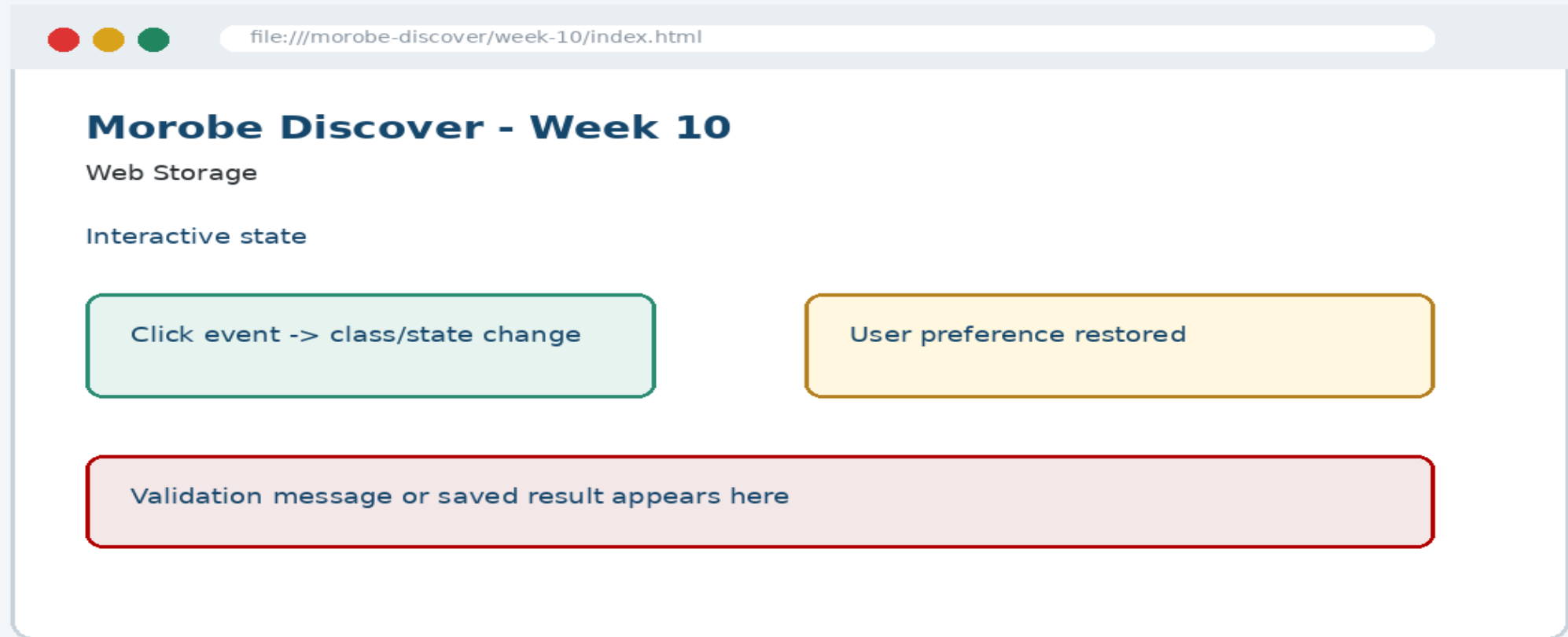
Predict the output

```
try {  
  const data = JSON.parse(savedValue);  
} catch {  
  localStorage.removeItem("favourites");  
}
```

Prediction prompt

What should change on screen when this code is applied correctly?

Browser result



Testing the feature

Refresh and reopen the browser
Inspect Application > Local Storage
Test corrupted data recovery
Verify sensitive data is not stored

Inspect with browser tools

Inspection path

Right click -> Inspect
Elements panel -> structure/classes
Console -> JavaScript errors
Network/Application -> data/storage when relevant

Evidence to save

Screenshot
Short test note
Changed files
Git commit message

Git evidence

```
git add .  
git commit -m "Week 10 Web Storage project increment"
```

Why it matters

Git history shows progression from Week 01 to final deployment. It is part of the assessment evidence.

Complete source code

Repository:
<https://github.com/YOUR-USERNAME/is229-morobe-discover>

Weekly folder:
/week-10/

Complete project:
/complete-project/

Reminder

Do not paste a whole file into a slide. Use the repository for complete code.

Mini challenge

Challenge

Modify one part of the Week 10 implementation and explain what changed in the browser.

State the file edited

State the line/section changed

Show the before and after result

What counts as evidence?

Technical evidence

Changed code files
Browser screenshot
Test note
Git commit

Understanding evidence

Code explanation
Why this method fits
Connection to previous week
Connection to next week

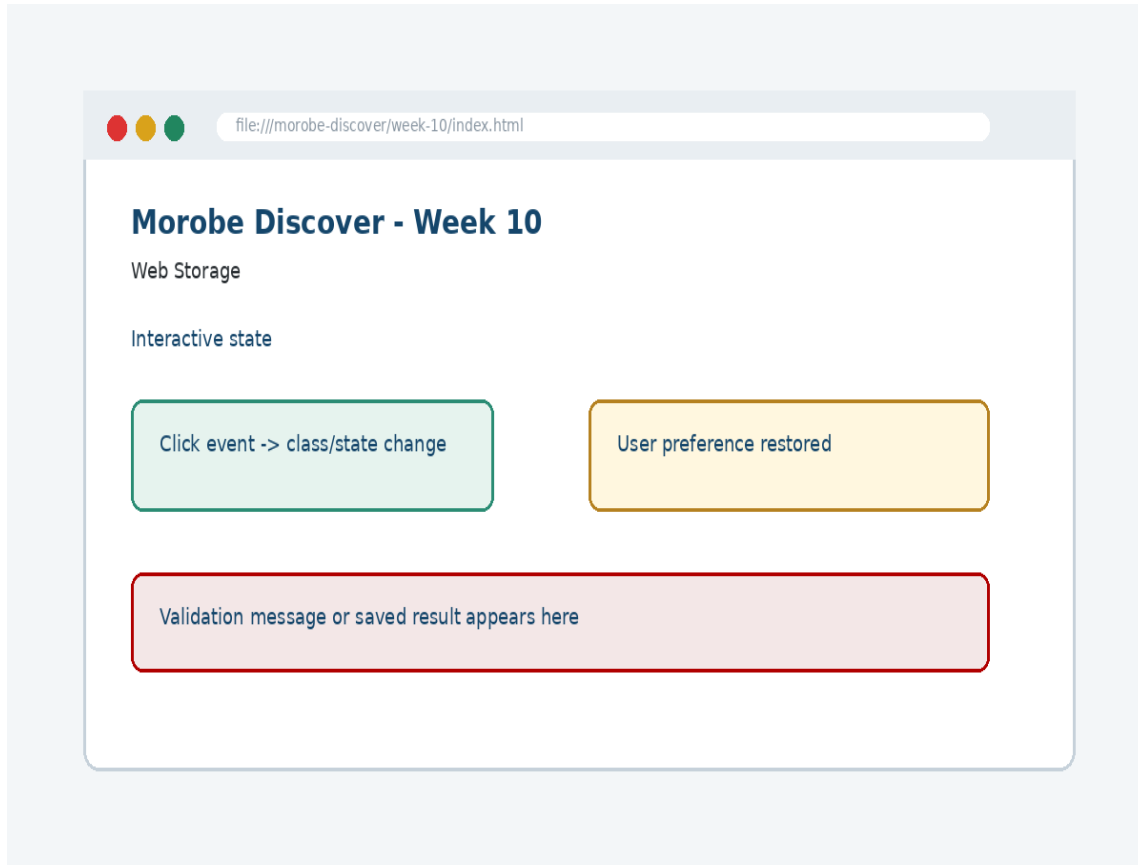
Project lab handoff

- Review current storage use
- Store favourite destination IDs
- Restore saved theme on page load
- Handle missing or corrupted storage safely
- Provide a clear reset option

Definition of done

Week 10 feature is implemented
Project still preserves earlier week features
Browser result matches the expected output
Test note and screenshot are saved
Git commit message is meaningful

What we built today



The site remembers non-sensitive user preferences and restores them predictably.
Evidence: localStorage for theme choice and saved IDs
Technology: Web Storage API

Lecture summary

Browser storage is useful for convenience, but it must be small, safe and recoverable.

localStorage persists after the browser closes

sessionStorage lasts only for a session

Testing and Git are part of development, not extras

Prepare for next class

Finish this week's lab before the next class
Read the next topic in advance
Bring the current project ready to extend
Be ready to explain one code decision

Exit question

How did Week 10 improve Morobe Discover, and what will the next week build on?